

Vectorless RAG Documentation

Vectorless RAG — Complete Project Documentation

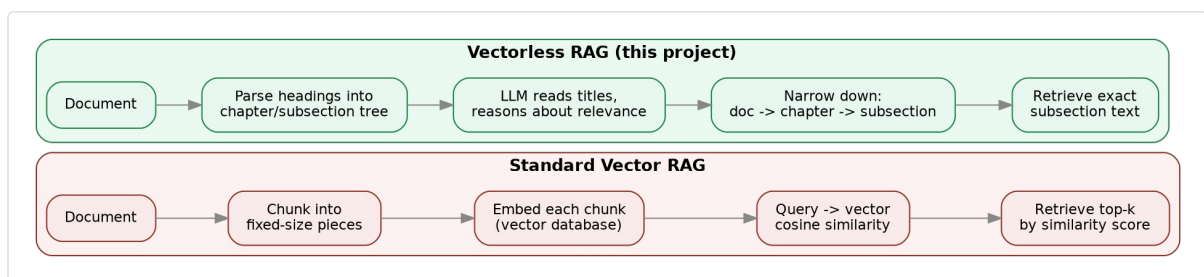
This documentation covers all current project files: **what's running, how it runs, why it's built this way, what's happening inside the code, which document types this technique is genuinely suited to, and whether hierarchical structure is required for it to work.**

1. What This Project Is

A retrieval-augmented generation (RAG) system that answers questions from documents **without embeddings or a vector database**. Instead of converting text into vectors and comparing similarity scores, it builds a table of contents from a document's headings and has an LLM *reason* its way to the right section — the way a person flips to a chapter title instead of scanning every page.

Why this approach exists: standard vector RAG retrieves by mathematical similarity, which can confidently retrieve the *wrong* section when two sections share vocabulary but differ in meaning (e.g., a clause that *creates* a warranty vs. one that *voids* it — both can score “similar” to a query about warranties, even though only one is correct). Tree-based navigation avoids this entirely, since it never compares meaning-as-numbers — it has an LLM read actual section titles and reason about relevance, narrowing down step by step.

Vector RAG vs. Vectorless RAG — Side by Side



Vector vs Vectorless Comparison

Both pipelines start from the same input (a document) and end with the same output (a retrieved piece of text) — but they get there in fundamentally different ways. The vector pipeline compares mathematical representations of meaning; the vectorless pipeline has the LLM reason directly over the document's own structure.

2. Project Files

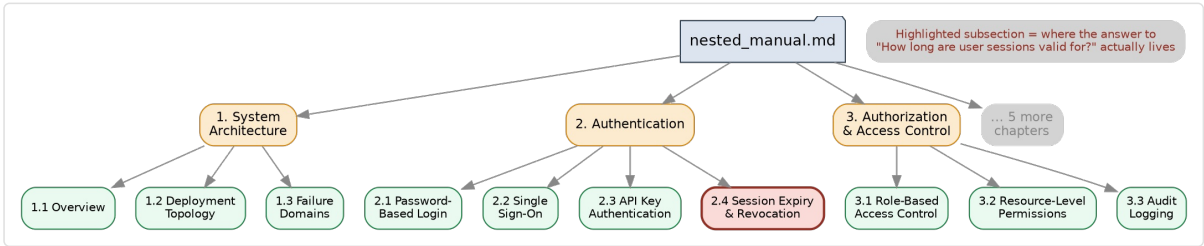
File	Role
<code>`app.py`</code>	The Streamlit application — full UI + pipeline, supports

	multiple documents via a document-selection hop
<code>`vectorless_rag.py`</code>	Standalone terminal/CLI script — runs a fixed list of test questions against <code>`nested_manual.md`</code> , prints the hop trail for each
<code>`requirements.txt`</code>	Python dependency list — <code>`langchain`</code> , <code>`langchain-groq`</code> , <code>`python-dotenv`</code> , <code>`streamlit`</code>
<code>`nested_manual.md`</code>	Sample document #1 — hand-written technical/infra manual, 8 chapters each with 3-4 subsections (fully nested, every chapter has subsections)
<code>`attention_paper.md`</code>	Sample document #2 — paraphrased structural mirror of “Attention Is All You Need,” 7 chapters, some flat (no subsections) and some nested

Relationship between ``app.py`` and ``vectorless_rag.py``: these are not duplicated effort. ``vectorless_rag.py`` is the original scratch/testing script — built first, step by step, with plain ``print()`` statements to verify each piece of the pipeline (parsing, navigation, answering) actually worked before anything else was layered on top. ``app.py`` came after, once that core logic was proven — it reuses the exact same functions (``load_tree``, ``pick``, ``answer``) but wraps them in a real Streamlit UI instead of terminal output. One is the “how the logic was verified,” the other is “the thing someone would actually run.”

3. Tree Structure — A Visual Example

Here is exactly what ``load_tree()`` builds when it parses ``nested_manual.md``, shown as an actual tree rather than a code dump. The highlighted node is where the answer to a real test question (“How long are user sessions valid for?”) is ultimately retrieved from:

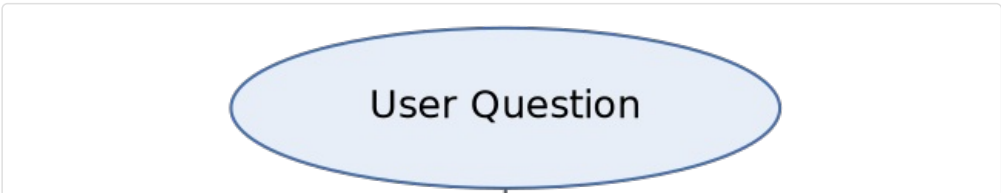


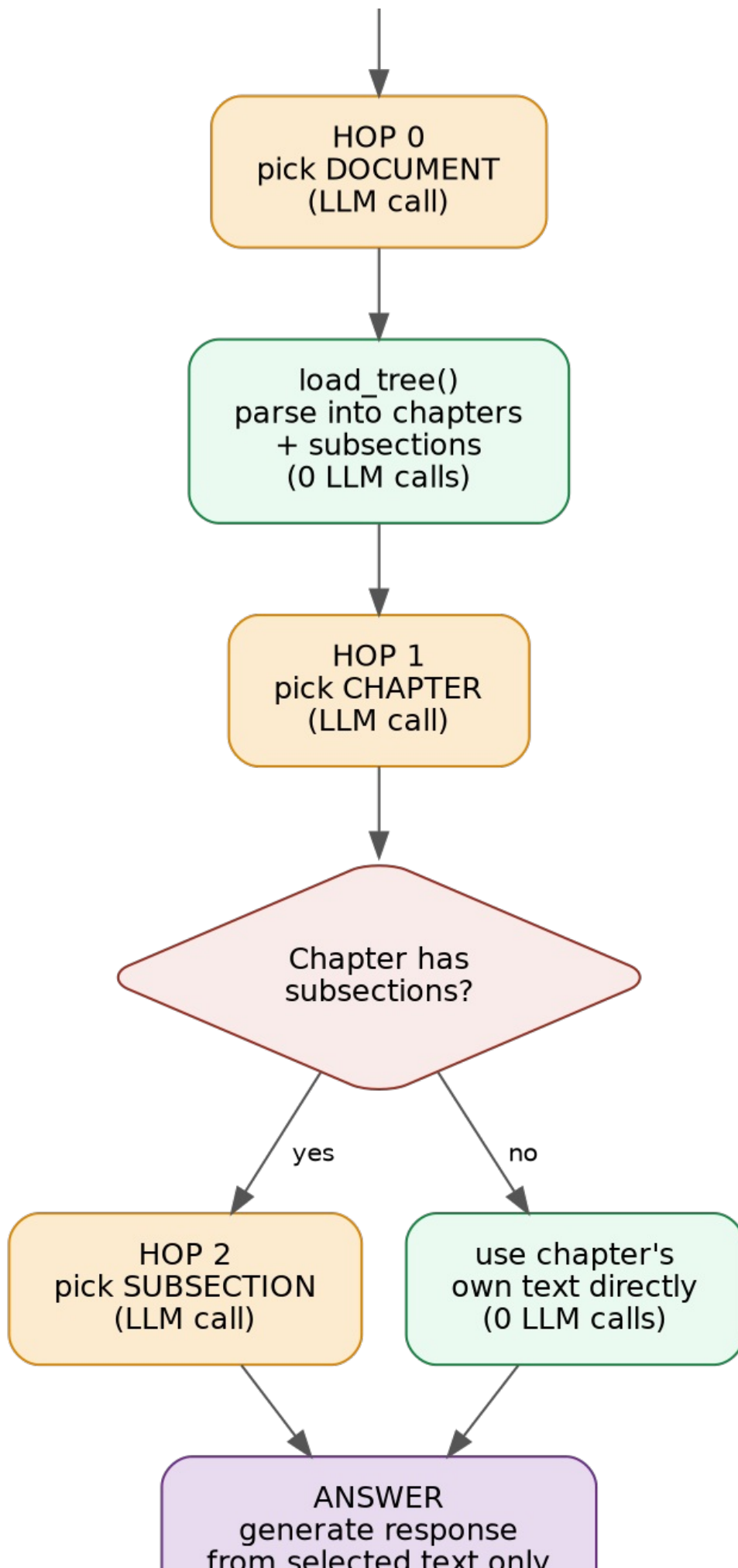
Tree Structure Diagram

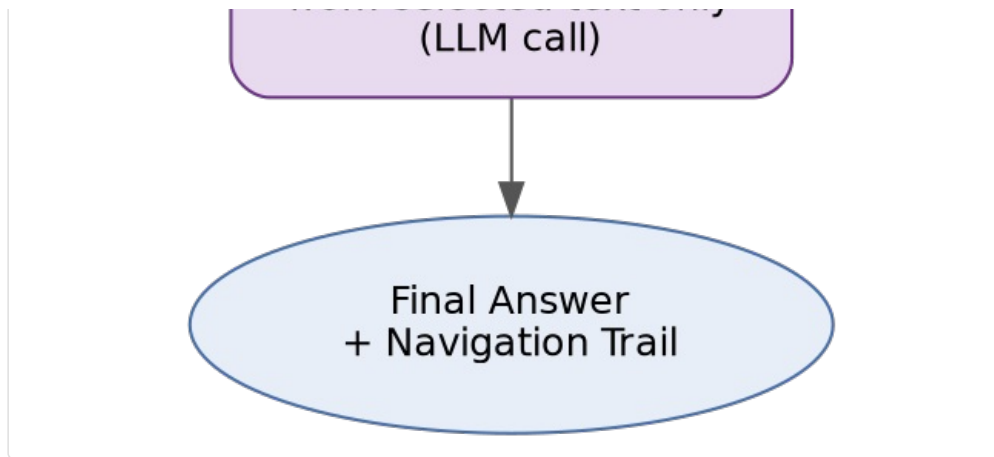
The document sits at the root. Chapters (orange) hang off it directly. Subsections (green) hang off their parent chapter — that nesting is exactly what ``chapter["subsections"].append(current_sub)`` produces in code. The highlighted subsection (red, “2.4 Session Expiry & Revocation”) is the exact leaf node the pipeline lands on after two navigation hops.

4. What’s Running, and Why: The Full Pipeline

Every question triggers **up to four LLM calls**, none of which touch a vector or an embedding:







Pipeline Flow Diagram

Why narrow in small steps instead of one big LLM call: each call is deliberately narrow — “pick 1 of N titles” — because that’s a far more reliable task for an LLM than “read this entire multi-thousand-word document and find the answer directly.” Progressive narrowing is the core mechanism this entire technique depends on.

Note the “Chapter has subsections?” branch point in the diagram — this is a real, necessary decision point, not a defensive afterthought. Four chapters in `attention\_paper.md` (Introduction, Background, Why Self-Attention, Conclusion) have no `##` subsections at all, so the pipeline must be able to answer directly from a chapter’s own text when there’s nothing to descend into further.

5. Code Walkthrough — What’s Happening Inside Each Function

`load\_tree(path)` — parsing a document into a tree (0 LLM calls)

```
```python def load_tree(path: str): with open(path, "r", encoding="utf-8") as f: raw = f.read()
```

```
 chapters = []
 current_chapter = None
 current_sub = None

 for line in raw.split("\n"):
 if line.startswith("# "):
 current_chapter = {"title": line[2:].strip(), "subsections": [], "text": ""}
 chapters.append(current_chapter)
 current_sub = None

 elif line.startswith("## "):
 current_sub = {"title": line[3:].strip(), "text": ""}
 current_chapter["subsections"].append(current_sub)

 else:
 if current_sub is not None:
 current_sub["text"] += line + "\n"
 elif current_chapter is not None:
 current_chapter["text"] += line + "\n"

 for chapter in chapters:
 chapter["text"] = chapter["text"].strip()
 for sub in chapter["subsections"]:
 sub["text"] = sub["text"].strip()

 return chapters
```

...

**Step by step:** 1. The entire file is read into one string, then split into individual lines. 2. Every line is classified as one of three things: a chapter heading (`#`), a subsection heading (`##`), or ordinary body text. 3. Two “bookmark” variables — `current\_chapter` and `current\_sub` — track where the parser currently is. A new chapter heading opens a new chapter box, which holds both a “subsections” list AND its own “text” field. A new subsection heading is placed *inside* the current chapter’s `subsections` list — that placement is what makes it “belong” to that chapter (visible directly in the tree diagram above). 4. Ordinary text lines get appended to whichever subsection is currently open. **If no subsection has been opened yet, the text goes onto the chapter’s own “text” field instead** — this is what lets a flat chapter (one with no `##` headings at all) still retain its content rather than losing it. 5. Returns a nested list: chapters, each holding a list of subsections (each with its own title/text) plus its own chapter-level text.

This function makes **zero LLM calls** — pure string processing, runs in milliseconds regardless of document length. This is the vectorless replacement for “build embeddings and populate a vector database.”

### **`pick(llm, index, question)` — the navigator (reused across every hop)**

```
python def pick(llm, index: str, question: str) -> int: prompt = PICK_PROMPT.format(index=index, question=question) response = llm.invoke([HumanMessage(content=prompt)]) raw = re.sub(r"^\s*json|$", "", response.content.strip(), flags=re.MULTILINE).strip() try: return int(json.loads(raw)["choice"]) except (json.JSONDecodeError, KeyError, ValueError): raise RuntimeError(f"Navigator returned something unparseable: {raw!r}")
```

Takes a numbered list of titles and the question, asks the LLM to reply strictly with `{“choice”: }`, and parses that integer out. The regex strips markdown code fences some models add despite instructions not to. If parsing fails, it raises an error rather than silently defaulting to a guessed index — a silent wrong guess would retrieve the wrong section and produce a confidently incorrect answer, which is worse than a visible failure.

This exact function is reused for every navigation decision shown in the pipeline diagram: picking the document (in `app.py`), picking the chapter, and picking the subsection — just with a different `index` string passed in each time.

### **The subsections check — handling chapters with no subsections**

```
python if chapter["subsections"]: s_idx = pick(llm, subsection_index(chapter), question) subsection = chapter["subsections"][s_idx] else: subsection = {"title": chapter["title"], "text": chapter["text"]}
```

This is the “Chapter has subsections?” decision point in the pipeline diagram. Without this check, the code would try to index into an empty list (`chapter["subsections"][s\_idx]` where the list has zero items), which throws `IndexError: list index out of range` — the LLM would still return some index like `0` for hop 2 since it has no way of knowing the list it was shown was empty. This check avoids that: if there are real subsections, do the normal hop 2; if not, use the chapter’s own text (captured correctly by `load\_tree`, per above) directly as the answer source.

### **`answer(llm, subsection, question)` — final generation (1 LLM call)**

```
python def answer(llm, subsection: dict, question: str) -> str: prompt = ANSWER_PROMPT.format(title=subsection["title"], text=subsection["text"], question=question) response = llm.invoke([HumanMessage(content=prompt)]) return response.content.strip()
```

By this point, the entire document has been narrowed to one subsection (or one flat chapter) — the final leaf node in the tree diagram. Only that text is sent here — nothing else from the document ever

reaches this call. The prompt instructs the model to answer **only** from that text and explicitly admit when the answer isn't there, rather than guessing — this is what prevents hallucination when navigation picks the wrong branch.

## ``run_pipeline` (`app.py`) / `run` (`vectorless_rag.py`) — orchestration`

Ties everything together in the exact order shown in the pipeline diagram. ``app.py``'s version adds the document hop at the start (``pick(llm, doc_index(), question)``) since it supports two documents; ``vectorless_rag.py``'s version skips straight to ``load_tree("data/nested_manual.md")``, since it's hardcoded to one file.

---

## 6. Which Document Types Work Well With This Approach

---

**Format is not the deciding factor. Structure is.** This technique can work with any file format, provided the underlying content has genuine hierarchy (chapters containing sections, sections containing subsections).

Format	Works well?	Why
Markdown (.md)	Yes — what's implemented here	Headings are plain-text markers ( <code>`#`</code> , <code>`##`</code> ) — trivial to detect with a string check.
PDF	Yes, in principle	The most common real-world target (contracts, filings, manuals) — but headings in a PDF are a <i>visual</i> property (font size, boldness), not a text marker. Extracting structure requires a layout-aware parser. <b>Not implemented in these files</b> — <code>`load_tree()`</code> only understands <code>`#`/`##`</code> text markers.
DOCX	Yes, in principle	Headings are a paragraph <i>style</i> attribute (e.g. <code>`Heading 1`</code> ), invisible to plain-text extraction. Requires <code>`python-docx`</code> , checking <code>`paragraph.style.name`</code> . <b>Not implemented in these files</b> either.
Plain .txt	Yes, if structured	Only works if the text follows some consistent heading convention the parser can detect.
CSV / tabular data	No — genuinely unsuited	Rows and columns have no hierarchy at all. This isn't a parser limitation — there's nothing for <i>any</i> implementation of this technique to build a tree from. SQL/pandas-style querying is the correct tool for this shape of data.
Unstructured text (chat logs, scraped pages, raw notes)	Poor fit	No table of contents exists to reason over. Vector-based RAG is the better tool here.

---

## 7. Direct Answer: Does This Require Hierarchical Data to Work Well?

---

**Yes — unambiguously. This is not a “works better with” case; it’s a hard dependency the entire mechanism is built on.**

Every hop in the pipeline (see the flow diagram in §4) depends on having a set of titles to choose from. Remove the hierarchy shown in the tree diagram in §3, and:

- **No headings at all:** `load_tree()` returns an empty list. There’s nothing for hop 1 to pick from — the pipeline would fail immediately, with no fallback available.
- **Inconsistent or shallow headings:** if a document’s headings don’t meaningfully group related content, the navigator is choosing between titles that don’t actually predict what’s inside them — the LLM is left guessing almost as blindly as with no structure at all, just with false confidence from having *some* titles to look at.
- **This project demonstrates a live example of partial structure being fragile:**  
`attention_paper.md` mixes nested chapters (Model Architecture, Training, Results) with flat ones (Introduction, Background, Why Self-Attention, Conclusion). Handling that mix correctly required the explicit fallback branch shown in the pipeline diagram — even *partial* absence of hierarchy needs special handling; full absence would leave no usable content at all.

**If your data doesn’t have real hierarchical structure, this approach will not work accurately — full stop.** In that situation, standard vector-based RAG (embeddings + similarity search) is the correct tool, not a fallback. The choice between vector RAG and this approach isn’t about which is generally “better” — it’s entirely determined by whether the source data has genuine structure to exploit. A flat document has nothing for this technique to navigate; a document with real chapters and sections, like the one visualized in §3, is exactly what it’s designed for.

---

## 8. Related But Distinct: GraphRAG

---

Worth being precise about this, since it’s easy to conflate: **GraphRAG (e.g. using Neo4j) is not a variant of this technique** — it solves a different-shaped problem entirely.

- **This project (vectorless RAG):** models structure *within* a single document. Answers “which section of this document covers X.”
- **GraphRAG:** models relationships *between* entities across a dataset (people, companies, transactions that reference each other), via a knowledge graph and Cypher queries. Answers “how is X connected to Y” (multi-hop reasoning), which this project’s tree-navigation cannot do at all.

GraphRAG wasn’t rendered obsolete by this approach, and this approach wasn’t built because GraphRAG was insufficient — building a full knowledge graph just to answer “which section of a manual covers refunds” would be significant, unnecessary overhead for a problem that has nothing to do with entity relationships. Vectorless RAG exists for a narrower, cheaper problem GraphRAG was never designed to solve in the first place.

---

## 9. Summary

---

This implementation demonstrates that tree-based, embedding-free retrieval is genuinely viable for structured documents — the two sample documents (one fully nested, one a realistic mix of nested and flat sections) confirm both the core mechanism and the edge case that partial structure requires explicit handling. The approach’s accuracy is entirely gated on the source document actually having meaningful

hierarchy; format (Markdown vs. PDF vs. DOCX) is a parsing detail that could be extended, not a limit on the technique itself.